

Theme: Creator Portfolio Builder

Section A: Conceptual Understanding

1. You are building a creator portfolio where profile data, projects, and social links render dynamically. Explain how JSX, component composition, and props work together to keep the UI reusable and consistent.

Ans. **JSX (JavaScript XML)** allows developers to write HTML-like code inside JavaScript. In a Creator Portfolio Builder, JSX is used to display profile information, projects, and social media links in a structured way.

Component Composition means dividing the UI into reusable components such as:

- Profile Component
- Project Component
- Social Links Component
- Header Component

Props are used to pass data from a parent component to child components. For example, creator information can be passed to the Profile component through props.

Together, JSX, component composition, and props make the UI reusable, organized, and consistent. If multiple creators use the same portfolio structure, only the data changes while the components remain the same.

2. The portfolio supports live updates when a creator edits details. Explain how state and event handling enable controlled updates without reloading the page.

Ans. **State** is used to store dynamic data that can change during the application's lifecycle. In a creator portfolio, profile details such as name, bio, or social links can be stored in state variables.

Event Handling allows the application to respond to user actions like:

- Clicking buttons
- Editing text fields
- Submitting forms

When a creator updates their profile information, an event handler captures the change and updates the state. React automatically re-renders the affected components, displaying the updated information instantly without refreshing the page.

This creates a smooth and interactive user experience.

3. Some sections should appear only if data exists (e.g., Projects). Explain how conditional rendering and list rendering manage this behavior efficiently.

Ans. **Conditional Rendering** is used to display components only when certain conditions are met.

Example:

- Show the Projects section only if projects are available.
- Display social media links only if the creator has added them.

This prevents empty sections from appearing on the page.

List Rendering is used when multiple items need to be displayed dynamically. For example, an array of projects can be rendered using the `map()` method to create project cards automatically.

Benefits:

- Efficient data display
- Reduced code duplication
- Easy management of dynamic content

Together, conditional and list rendering help create a clean and responsive portfolio interface.

4. Profile data should persist across reloads. Explain how LocalStorage or API-based persistence integrates with component state.

Ans. Profile data should remain available even after the browser is refreshed.

LocalStorage

LocalStorage stores data in the user's browser.

Process:

1. Save profile data whenever it changes.
2. Retrieve the saved data when the application loads.
3. Initialize component state with the stored data.

Benefits:

- Simple implementation
- No server required
- Data persists across page reloads

API-Based Persistence

An API can store creator data on a server or database.

Process:

1. Send updated profile data to the server.
2. Store the data in a database.
3. Fetch the data when the application starts.
4. Update component state with the retrieved data.

Benefits:

- Accessible from multiple devices
- Better security
- Suitable for large-scale applications

5. As the app grows, explain how breaking the UI into smaller components improves maintainability and performance.

Ans. As the application grows, managing a large single component becomes difficult.

Breaking the portfolio into smaller components provides:

Better Maintainability

Each component handles a specific responsibility, making code easier to understand and modify.

Reusability

Components such as buttons, cards, and profile sections can be reused throughout the application.

Easier Testing

Individual components can be tested independently.

Improved Collaboration

Different developers can work on different components simultaneously.

Better Performance

React updates only the components whose data changes, reducing unnecessary rendering.

This approach results in cleaner, more organized, and efficient code.

6. Explain how this component-based architecture prepares the app for future scaling or migration to [Next.js](#).

Ans. A component-based architecture makes future expansion easier.

Scalability

New features such as:

- Blogs
- Testimonials
- Contact Forms
- Analytics Dashboard

can be added as separate components without affecting existing functionality.

Code Reusability

Existing React components can be reused across different pages and modules.

Easy Migration to Next.js

Since Next.js is built on React, most components can be moved with minimal changes.

Benefits of migration:

- Server-Side Rendering (SSR)
- Static Site Generation (SSG)
- Better SEO
- Faster page loading
- Improved performance

Therefore, a well-structured component-based architecture not only improves current development but also prepares the Creator Portfolio Builder for future growth and advanced React frameworks like Next.js.

Section B: Mini Tasks - Creator Portfolio Builder

Task: Portfolio Item Manager

1. Reusable ProjectCard Component: Build a component that displays a Project's Title, Description, and Year. Ensure the styling is encapsulated so it looks consistent regardless of the content.

Ans. **ProjectCard.jsx**

```
import React from "react";
```

```
function ProjectCard({ title, description, year }) {
  return (
    <div className="project-card">
      <h2>{title}</h2>
      <p>{description}</p>
      <span>Year: {year}</span>
    </div>
  );
}
```

```
export default ProjectCard;
```

App.css

```
.project-card {
```

```

border: 1px solid #ccc;
padding: 15px;
margin: 10px 0;
border-radius: 8px;
background-color: #f9f9f9;
box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.project-card h2 {
  margin-bottom: 10px;
}

.project-card p {
  margin-bottom: 10px;
}

```

2. Manage State: Create a stateful array called projects. Build a small form that allows a user to input a new project's details and add it to the list.

Ans. **App.jsx**

```

import React, { useState } from "react";
import ProjectCard from "../ProjectCard";
import "../App.css";

function App() {
  const [projects, setProjects] = useState([
    {
      title: "Portfolio Website",
      description: "Personal portfolio built with React.",
      year: 2025
    }
  ]);

  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [year, setYear] = useState("");

  const addProject = (e) => {
    e.preventDefault();

```

```
const newProject = {
  title,
  description,
  year
};

setProjects([...projects, newProject]);

setTitle("");
setDescription("");
setYear("");
};

return (
  <div className="container">
    <h1>Creator Portfolio Builder</h1>

    <form onSubmit={addProject}>
      <input
        type="text"
        placeholder="Project Title"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        required
      />

      <input
        type="text"
        placeholder="Project Description"
        value={description}
        onChange={(e) => setDescription(e.target.value)}
        required
      />

      <input
        type="number"
        placeholder="Year"
        value={year}
        onChange={(e) => setYear(e.target.value)}
        required
      />

      <button type="submit">Add Project</button>
    </form>
  </div>
);
```

```

    <div>
      {projects.map((project, index) => (
        <ProjectCard
          key={index}
          title={project.title}
          description={project.description}
          year={project.year}
        />
      ))}
    </div>
  </div>
);
}

```

```
export default App;
```

Additional CSS

```

.container {
  width: 60%;
  margin: auto;
  padding: 20px;
}

```

```

form {
  margin-bottom: 20px;
}

```

```

input {
  display: block;
  width: 100%;
  padding: 8px;
  margin: 10px 0;
}

```

```

button {
  padding: 10px 15px;
  cursor: pointer;
}

```


3. Render Lists using map(): Display the projects array in a grid layout. Each item in the array must be rendered using your ProjectCard component.
Requirement: Every list item must have a unique key.

Ans. The `projects` array can be displayed in a grid layout using the `map()` method. Each project is rendered through the reusable `ProjectCard` component. A unique `id` is used as the key.

Rendering Projects

```
<div className="project-grid">
  {projects.map((project) => (
    <ProjectCard
      key={project.id}
      title={project.title}
      description={project.description}
      year={project.year}
    />
  ))}
</div>
```

Grid CSS

```
.project-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin-top: 20px;
}
```

Adding Unique IDs

```
const newProject = {
  id: Date.now(),
  title,
  description,
  year
};
```

4. Conditional Rendering (Empty State): If the projects array is empty, hide the grid and show a "Getting Started" callout: "You haven't added any projects to your portfolio yet!"

Ans. If there are no projects in the portfolio, hide the project grid and display a "Getting Started" message.

JSX

```
{
  projects.length === 0 ? (
    <div className="empty-state">
      <h3>Getting Started</h3>
      <p>You haven't added any projects to your portfolio yet!</p>
    </div>
  ) : (
    <div className="project-grid">
      {projects.map((project) => (
        <ProjectCard
          key={project.id}
          title={project.title}
          description={project.description}
          year={project.year}
        />
      ))}
    </div>
  )
}
```

CSS

```
.empty-state {
  text-align: center;
  padding: 30px;
  border: 2px dashed #ccc;
  border-radius: 10px;
  background: #f8f8f8;
}
```

5. Persist Data Locally: Ensure the project list is saved to localStorage. Implement a "Clear Portfolio" button that wipes the state and the local storage simultaneously.

Ans. Use `localStorage` to save projects so they remain available after page refreshes.

Import Hooks

```
import React, { useState, useEffect } from "react";
```

Load Data from localStorage

```
const [projects, setProjects] = useState(() => {  
  const savedProjects = localStorage.getItem("projects");  
  
  return savedProjects  
    ? JSON.parse(savedProjects)  
    : [];  
});
```

Save Data Whenever Projects Change

```
useEffect(() => {  
  localStorage.setItem(  
    "projects",  
    JSON.stringify(projects)  
  );  
}, [projects]);
```

Clear Portfolio Button

This button removes all projects from both React state and localStorage.

Function

```
const clearPortfolio = () => {  
  setProjects([]);  
  localStorage.removeItem("projects");  
};
```

Button

```
<button onClick={clearPortfolio}>  
  Clear Portfolio  
</button>
```